CyberSentinel: Big Data-Based Network Intrusion Detection Using Hadoop HDFS, PySpark and Random Forest

Submitted by

Inchara M P (2023BCS0129)

Aysha Lintha (2023BCS0183)

Guided by

Dr. John Paul Martin

Indian Institute of Information Technology Kottayam

1. Introduction

An intrusion detection system (IDS) watches network traffic and raises an alert when it sees something suspicious, such as a denial-of-service flood, repeated login attempts, botnet messages or a web attack. Most machine-learning IDSs work on network flows: each connection is summarised by numbers like its duration, how many packets it sent, how long those packets were and how much time passed between them [1, 2]. The public datasets used to train these models are big. CSE-CIC-IDS2018, the one we use, has more than sixteen million flows spread over ten CSV files [3, 2], so just storing and cleaning it is a data-engineering task before any model is trained.

Our project, **CyberSentinel**, is a batch Big Data pipeline for this problem. We keep the processed CSE-CIC-IDS2018 files in Hadoop HDFS [4], clean them and turn them into feature vectors with PySpark [5], and label every flow as Benign or Attack with a Random Forest from Spark MLlib [6, 7]. A Streamlit dashboard shows the results and can classify a CSV file that a user uploads, using the model stored in HDFS. All of this runs on one Ubuntu virtual machine with 8 GB of RAM, so it should be read as a Big Data prototype and not as a multi-node cluster.

A high accuracy on a random split of a public dataset does not tell us much about how a detector will cope with attacks it was never trained on [8, 9]. So, along with the usual evaluation, we carried out three extra analyses:

- a **data-quality audit** of each of the ten CSV files, which records the schema differences, repeated header rows and invalid values behind every cleaning step;
- an **attack-family breakdown** of the attacks the model misses, to see which family the false negatives come from;
- a **known-versus-unseen test**, where we leave one attack family out of training at a time and compare the Random Forest with a K-Means anomaly detector trained only on normal traffic, and with a hybrid of the two, at false-alarm rates set on separate normal data.

We also repeated our timing benchmark to see how much the prediction speed changes from run to run. The rest of the report is organised as follows: Section 2 reviews related work, Section 3 sets out the problem and our research questions, Section 4 explains the architecture and methods, Section 5 presents and discusses the results, and Section 6 concludes. The code and a step-by-step guide to reproduce everything are in our GitHub repository, `https://github.com/idsaturn07/CyberSentinel` [10].

2. Literature Survey

2.1. Intrusion-detection datasets and their quality

Sharafaldin, Lashkari and Ghorbani explained how the CIC datasets are built: normal background traffic is generated from user profiles, attacks are launched from a separate network, and CICFlowMeter turns the captured traffic into flows with more than 80 features [1]. CSE-CIC-IDS2018 was made the same way on a larger AWS testbed. It is released as raw packet captures and as processed CSV files, one for each day of capture [3]. In their survey of network IDS datasets, Ring et al. point out that things like how the data was labelled, how long it was recorded and how balanced the classes are have a big effect on what a model trained on it can really show [11].

Other researchers have looked closely at the quality of the CIC data itself. Engelen, Rimmer and Joosen found mistakes in both the labels and the way flows were built in CICIDS2017 [12]. Liu et al. did a similar check on CIC-IDS-2017 and CSE-CIC-IDS-2018, found problems in how attacks were run, how features were generated and how flows were labelled, and released corrected versions of both [13]. Leevy and Khoshgoftaar's survey also notes that many papers using CSE-CIC-IDS2018 do not describe their data cleaning well enough for someone else to repeat it [2]. These findings are the reason we audited the files ourselves and wrote down exactly what we removed or changed, and why.

2.2. Machine learning on CSE-CIC-IDS2018

Leevy and Khoshgoftaar's review of models built on CSE-CIC-IDS2018 warns that some very high reported scores may be a sign of overfitting, that class imbalance is often ignored, and that results are hard to compare between papers [2]. Karataş, Demir and Şahingöz dealt with the imbalance by oversampling the rarer attack classes and compared several classifiers, giving results for each attack class [14]. Songma, Sathuphan and Pamutha combined cleaning, normalisation and principal component analysis with several classifiers and got strong results with XGBoost, decision trees and Random Forest [15]. Random Forest was introduced by Breiman: it trains many randomised decision trees and lets them vote, and it also tells us how important each feature was [6].

2.3. Big Data platforms for intrusion detection

HDFS splits large files into blocks, keeps copies of them, and uses a NameNode to track where each block is; it is built for reading large files quickly from start to end [4, 16]. Apache Spark keeps intermediate results in memory and offers a DataFrame API [5]. On top of it, MLlib provides scalable classifiers such as Random Forest, along with tools for building features and measuring results [7, 17]. Several groups have used Spark for intrusion detection before. Othman et al. built Spark-Chi-SVM, which selects

features with a chi-squared test and classifies with an SVM, and tested it on KDD99 [18]. Atefinia and Ahmadi compared six MLlib algorithms on the botnet part of CSE-CIC-IDS2018 using one to eight worker machines; adding workers cut training time, while the detection scores hardly changed [19]. Mamdouh et al. built a Spark pipeline on CSE-CIC-IDS2018 that ranks features using Random Forest importance and then classifies with a weighted Random Forest, reaching a weighted F1-score of about 0.999 with 34 features [20]. So using a Random Forest on Spark for this dataset is not new, and we do not claim it as our contribution.

2.4. Generalisation and anomaly detection

Sommer and Paxson argued that machine learning in intrusion detection is good at spotting things that look like what it has already seen, but much weaker at spotting truly new attacks, and that evaluations should be designed with this in mind [8]. Arp et al. listed common mistakes in security machine learning, such as biased sampling and accidentally letting test information leak into training, which give scores that do not hold up in practice [9]. Cantone, Marrocco and Bria trained four classifiers on four IDS datasets, one of them a corrected CSE-CIC-IDS2018. The models were nearly perfect when tested on the same dataset they were trained on, but across datasets they mostly did no better than guessing [21].

Anomaly detection works the other way round. Instead of learning what attacks look like, it learns what normal traffic looks like and flags anything far from it, so in principle it can catch attacks it has never seen [22]. Kitsune is one example: it trains a group of autoencoders on normal traffic and uses them to detect attacks as they happen [23]. A simpler option is to cluster normal traffic; Spark MLlib includes K-Means with the scalable k-means|| starting method [24, 7]. Table 1 lists the studies closest to our work.

Table 1: Summary of the most closely related work.

Study	Platform / method	Data	Relation to this work
Leevy & Khoshgoftaar [2]	Survey	CSE-CIC-IDS2018	Overfitting, imbalance and reproducibility concerns
Liu et al. [13]	Dataset error analysis	CIC-IDS-2017/2018	Documents data errors; motivates the audit
Karataş et al. [14]	Oversampling + ML	CSE-CIC-IDS2018	Per-class results on the same dataset
Atefinia & Ahmadi [19]	Six Spark MLlib models	Botnet subset	Spark on CSE-CIC-IDS2018, multi-worker timing
Mamdouh et al. [20]	Spark + weighted RF	CSE-CIC-IDS2018	Closest prior work: same platform and model
Cantone et al. [21]	Four classifiers	Four datasets	Cross-dataset failure of high in-dataset scores
This project	HDFS + Spark RF, K-Means, hybrid	Nine files, 8.28 M flows	Audit, missed-attack analysis, unseen families

Putting these together, Spark-based Random Forest detection, class imbalance and generalisation have each already been studied on this dataset. Our question is narrower. Inside one reproducible Spark pipeline running on modest hardware, we ask how the dataset's quality problems, the way missed attacks are spread across families, and the detection of left-out families change what the headline accuracy really means. As far as we know, this combination has not been reported for CSE-CIC-IDS2018, although a wider literature search would be needed to be sure.

3. Problem Statement

Network-flow datasets like CSE-CIC-IDS2018 are too large to handle comfortably with ordinary single-process tools, and their CSV files contain mismatched headers, repeated header rows and values that are not valid numbers. Scores reported on this data are often hard to judge because papers do not explain their cleaning, class balance or errors per class [2, 13]. A random train/test split adds a further problem: the attacks in the test set come from the same families as those in training, so the test only checks whether the model recognises attacks it already knows [8]. Our aim is therefore to build a traceable HDFS and PySpark pipeline that cleans and classifies this data, and to test it in a way that shows both where the classifier works and where it fails. We focus on four research questions:

RQ1. How can we find, handle and document the schema differences, repeated headers and invalid values in the processed files in a way others can repeat?

RQ2. How well does a Spark Random Forest separate normal from attack traffic on a held-out test set, and which attack families make up the attacks it misses?

RQ3. How well does the Random Forest detect an attack family that was left out of its training, and can an anomaly detector trained only on normal traffic, alone or together with the Random Forest, recover those attacks at an acceptable false-alarm rate?

RQ4. How do prediction time and throughput change with batch size on a single-machine Spark set-up, and how repeatable are these timings?

The project covers offline, two-class (Benign/Attack) classification on one virtual machine. Live packet capture, streaming, multi-class labelling and production deployment are outside its scope. Leaving a family out of a public dataset imitates a new attack type, but it is not the same as testing against a real zero-day attack.

4. Proposed Solution / Architecture

4.1. System architecture

Figure 1 shows the whole pipeline. The processed CSV files are copied into HDFS, read and cleaned by PySpark, combined into a 78-value feature vector, and used to train the Random Forest. The cleaned data is saved back to HDFS as Parquet, and every later step (training, evaluation, cross-validation and benchmarking) reads this Parquet copy rather than parsing the CSV files again. The trained model is saved both on the local disk and in HDFS, and the Streamlit dashboard loads it from HDFS to make predictions on uploaded CSV files. Metrics and figures go into a results folder that both the dashboard and this report use.

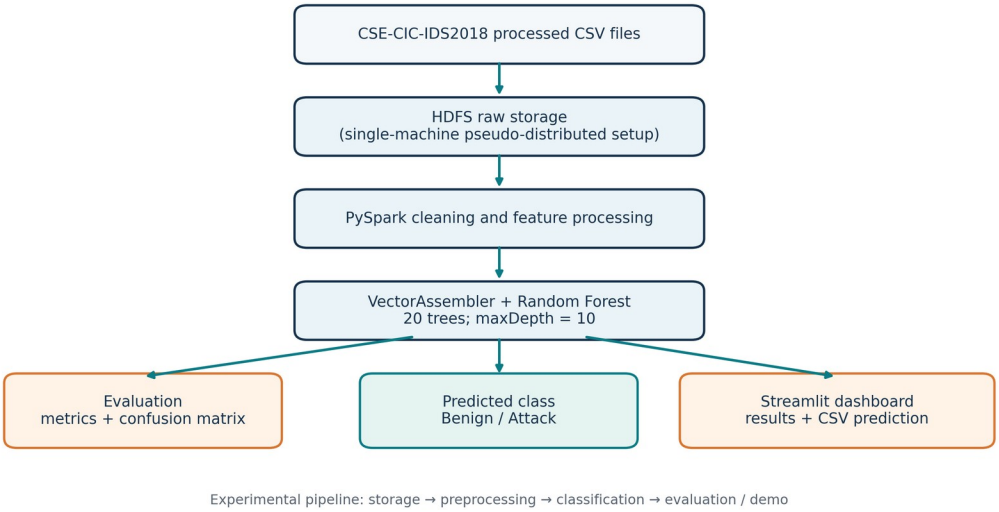


Figure 1: CyberSentinel pipeline: HDFS storage, PySpark preprocessing, Random Forest classification, and evaluation and dashboard outputs.

4.2. Dataset and preprocessing

Nine of the ten processed CSE-CIC-IDS2018 files have the same 80-column header, and these nine are used for modelling. We left out the Tuesday 20-02-2018 file because its header has 84 columns (see Section 5.1). Each record holds a timestamp, the original text label and 78 numeric flow features. During preprocessing we drop rows whose `Label` field just repeats the header text, set Benign to 0 and every other label to 1, replace missing, NaN and ±Infinity values in the 78 features with zero, and combine the features with `VectorAssembler`. The data is then split at random into 80% training and 20% test, with seed 42. Table 2 gives the counts at each step.

Table 2: Record counts through preprocessing.

Stage	Records	Note
Rows read from the nine files	8,284,254	Spark CSV read
Repeated header rows removed	59	Label = "Label"
Cleaned working dataset	8,284,195	6,112,151 benign (73.8%), 2,172,044 attack (26.2%)
Training partition	6,628,175	80%, seed 42
Held-out test partition	1,656,020	20%, seed 42

4.3. Data-quality audit

A separate script, `data_quality_audit.py`, opens each of the ten files on its own, using that file's own header and reading every value as text, so that the literal strings `NaN` and `Infinity` are counted exactly. For every file it notes the number of columns and any that differ from the shared schema, how many rows were read, how many repeated header rows there are, the missing, NaN and Infinity values in each feature, the labels found, and whether the file is used. The results are saved as CSV and JSON files, so each cleaning decision is backed by a written record.

4.4. Random Forest model and evaluation

We use Spark MLlib's `RandomForestClassifier` with 20 trees, a maximum depth of 10 and seed 42. We picked these settings so that training fits in the 8 GB machine, not through a parameter search. The model is trained on the training split and tested once on the held-out test split. Spark's evaluator gives the overall accuracy and the weighted precision, recall and F1-score. We work out attack precision, attack recall, attack F1-score and the false-negative rate (FN / (FN + TP)) from the confusion matrix ourselves, because a high overall score can hide missed attacks. We also measure training accuracy, and run a three-fold `CrossValidator` on a 500,135-record sample of the training split to check that the result is stable.

4.5. Known versus unseen attack families

The nine files contain 13 attack labels, which we grouped into six families (Table 3). The text label is used only for this grouping and is never given to the model as an input. We split the cleaned data once, with seed 42, into a 70% training pool, a 10% calibration set (of which we use only the normal rows) and a 20% test set. This test set holds exactly the same 1,220,899 benign and 435,121 attack records as the 20% test split in Section 4.2, because Spark draws the same random number for each row when the data and seed are the same.

Table 3: Attack families and their record counts in the test set *T* (main evaluation) and in the training pool plus test set (robustness check).

Family	Labels in the dataset	Test (T)	Pool + test
Bot	Bot	57,522	257,577
Brute-force	FTP-BruteForce, SSH-Bruteforce	76,112	342,873
DDoS	DDOS attack-HOIC, DDOS attack-LOIC-UDP	137,982	619,073
DoS	DoS attacks-Hulk, -SlowHTTPTest, -GoldenEye, -Slowloris	130,705	588,688
Infiltration	Infilteration	32,611	145,744
Web	Brute Force -Web, Brute Force -XSS, SQL Injection	189	831

For each family we train a Random Forest, with the settings from Section 4.4, that never sees a single record of that family. We also train a reference forest on all families. All four detectors are then compared on **the same records**, namely that family's records in the test set *T*. The reference forest gives the “seen” recall, the forest trained without the family gives the “unseen” recall, and the anomaly detector and the hybrid are scored on the very same rows. Since the forest trained without a family has not used any of that family's pool records either, we also measure its unseen recall on all of the family's records in $P \cup T$, a larger sample, as a robustness check. We record exact detected counts for every detector. To fit in memory, each forest is trained on a random sample of at most 1,000,000 rows from the pool, but testing always uses the full data. For the anomaly detector we standardise the features and fit K-Means with k = 10 on 200,000 normal training rows; a flow's anomaly score is its squared distance to the nearest cluster centre. We set two thresholds at the 99th and 99.9th percentiles of this score on the 611,462 normal calibration rows, which aims for false-alarm rates of 1% and 0.1%. The thresholds are never adjusted using test data. The hybrid flags a flow if either the Random Forest or the anomaly detector flags it. Algorithm 1 sums up the procedure.

Algorithm 1 Known-versus-unseen evaluation of the Random Forest (RF), anomaly detector (AD) and hybrid

Split cleaned data (seed 42) into pool P (70%), calibration C (10%), test T (20%)
$s \leftarrow$ StandardScaler fitted on benign rows of P; $K \leftarrow$ K-Means(k = 10) on 200,000 scaled benign rows
score(x) $\leftarrow$ min over centres c of $\|s(x) - c\|^2$
$\tau_1 \leftarrow$ 99th percentile, $\tau_2 \leftarrow$ 99.9th percentile of score on benign rows of C
$RF_{all} \leftarrow$ RandomForest(sample $\leq$ 1 M rows of P); record seen recall per family on T
for each family f in {Bot, Brute-force, DDoS, DoS, Infiltration, Web} **do**
    RF–f $\leftarrow$ RandomForest(sample $\leq$ 1 M rows of P with family f removed)
    $E \leftarrow$ records of f in T (main result); repeat with $P \cup T$ (robustness check)
    for τ in $\{\tau_1, \tau_2\}$ **do**
        AD(x) $\leftarrow$ score(x) > τ; Hybrid(x) $\leftarrow$ RF–f(x) **or** AD(x)
        record recall of RF–f, AD and Hybrid on E
    end for
end for
Report false-alarm rates of RF_{all}, AD and Hybrid on the benign rows of T

4.6. Prediction benchmark

For the timing test, we save a one-million-record subset of the cleaned feature data to disk and first run a 10,000-record warm-up prediction. Then, for 100,000, 250,000, 500,000 and 1,000,000 records, the saved model predicts the batch, the predictions are grouped and counted by class, and the counts are collected, while we record the wall-clock time and records per second. Our first benchmark timed each size once. In the repeated benchmark each size is run three times in rotation, so that if the machine slows down it affects every size equally, and we report the mean and standard deviation.

4.7. Implementation

The system runs on Ubuntu 24.04 in VirtualBox (8 GB RAM, 4 vCPUs) with Java 17, Hadoop 3.3.6 in single-node mode (replication 1), Python 3.12 and PySpark 4.2 in local mode with 3 GB of driver memory. In HDFS, the raw files live under `/cybersecurity/CSE_CIC_IDS2018/raw/` and the model under `.../results/random_forest_model`. A shared module, `common.py`, holds the file list, the Spark session and the cleaning code. The steps are run in this order: `data_preprocessing.py`, `dataset_statistics.py`, `data_quality_audit.py`, `train_model.py`, `evaluate_model.py`, `cross_validation.py`, `scalability_experiment.py`, `generate_plots.py` and `unseen_attack_experiment.py`. Because HDFS is the Spark session's default file system, we pass local paths to Spark as explicit `file://` URIs.

The Streamlit dashboard (`app.py`) has six pages: Dashboard Overview, Model Evaluation, Performance Analysis, Data Quality Audit, Known vs Unseen Attacks, and CSV Anomaly Detection. On the last page a user can upload a CSV with the 78 feature

columns; the app puts the columns in training order, replaces invalid values with zero, loads the model from HDFS, and then shows and exports the predictions. The dataset itself is not in the repository, but the README and `data/README.md` give the commands to download and set it up [10, 3].

5. Results & Analysis

5.1. Data-quality audit (RQ1)

Table 4 summarises our audit of the ten files. Nine of them share the same 80-column header. The Tuesday 20-02-2018 file has 84 columns, because it has four extra identifier columns (Flow ID, Src IP, Src Port, Dst IP) that push every following column out of place. This is why Spark showed a schema warning when all the files were read together, with a value such as `Idle Mean` turning up where `Label` should be. In other words, the file is not corrupt; it simply does not match the shared schema.

Table 4: Per-file audit of the ten processed CSV files.

Capture day	Cols	Rows read	Rep. headers	NaN	Infinity	Decision
Fri 02-03-2018	80	1,048,575	0	2,558	5,542	Included
Fri 16-02-2018	80	1,048,575	1	0	0	Included
Fri 23-02-2018	80	1,048,575	0	3,754	7,662	Included
Tue 20-02-2018	84	7,948,748	0	36,767	82,139	Excluded
Thu 01-03-2018	80	331,125	25	1,834	4,004	Included
Thu 15-02-2018	80	1,048,575	0	4,921	11,133	Included
Thu 22-02-2018	80	1,048,575	0	3,569	7,651	Included
Wed 14-02-2018	80	1,048,575	0	2,277	5,371	Included
Wed 21-02-2018	80	1,048,575	0	0	0	Included
Wed 28-02-2018	80	613,104	33	4,041	8,297	Included

The excluded file is a big one: it holds 7,948,748 of the 16,233,002 rows across all ten files (49.0%), including 7,372,557 normal flows and all 576,191 DDoS-LOIC-HTTP records, so that attack type does not appear in any of our experiments. Repeated header rows showed up in three files (1, 25 and 33 rows), which explains the 59 rows removed in Table 2. No feature had blank values. Invalid values appeared in only two of the 78 features, both of them per-second rates: Flow Byts/s (22,954 NaN and 13,353 Infinity) and Flow Pkts/s (36,307 Infinity). Each has exactly 36,307 invalid entries (0.44% of the working rows), which fits with flows whose recorded duration is zero. We replace these values with zero; this is simple, but it may distort these two features.

Two more findings limit how far our results can be taken. Seven of the nine files we used have exactly 1,048,575 data rows, one fewer than the maximum number of rows in a Microsoft Excel sheet, which hints that they may have been cut short before release; the audit cannot prove this. Each attack type was also recorded on particular days, for

example Infiltration on 28-02 and 01-03 and the web attacks on 22-02 and 23-02, so leaving out a family in Section 5.4 also leaves out part of those days' traffic.

5.2. Overall classification results (RQ2)

Table 5 gives the Random Forest's results on the 1,656,020-record held-out test set, and Figure 2 shows the confusion matrix. Test accuracy was 97.66%, very close to the training accuracy of 97.68% and the three-fold cross-validation accuracy of 97.63%, so accuracy gives no sign of serious overfitting on this split. Attack precision (98.55%) is clearly higher than attack recall (92.47%). In practice, when the model says a flow is an attack it is nearly always right, but it labelled 32,779 attack flows (7.53%) as normal, compared with 5,907 normal flows wrongly flagged as attacks.

Table 5: Random Forest results on the held-out test set. *Mean over three folds of a 500,135-record training sample.

Metric	Value	Metric	Value
Test accuracy	97.66%	Attack precision	98.55%
Weighted precision	97.68%	Attack recall	92.47%
Weighted recall	97.66%	Attack F1-score	95.41%
Weighted F1-score	97.64%	False-negative rate	7.53%
Training accuracy	97.68%	3-fold CV accuracy*	97.63%

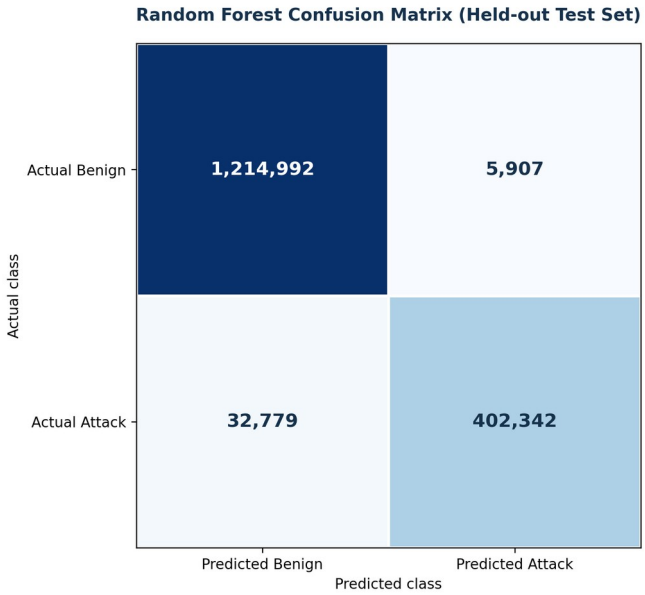


Figure 2: Confusion matrix on the 1,656,020-record held-out test set (TN 1,214,992; FP 5,907; FN 32,779; TP 402,342).

Figure 3 shows the fifteen most important features. The top five are maximum packet length (0.082), destination port (0.071), minimum forward segment size (0.067), maximum flow inter-arrival time (0.055) and forward packets per second (0.049). These scores show what the trained forest relies on for this dataset. They do not mean that these features cause attacks, or that they would rank the same way on other data.

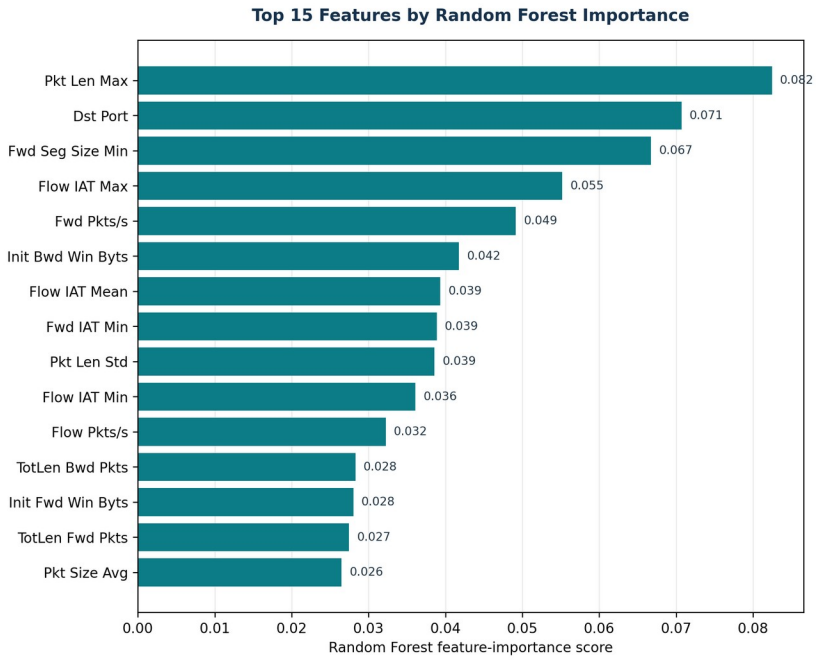


Figure 3: Top 15 Random Forest feature-importance scores.

5.3. Which attacks are missed

Table 6 splits the test results of the reference forest from Section 4.5 by attack family. Bot, Brute-force, DDoS and DoS are caught almost every time. Infiltration is the exception: even though it is in the training data, only 0.65% of it is detected, and it makes up 32,398 of the 32,814 missed attacks, or 98.7% of all false negatives. In the 78 flow features, Infiltration traffic looks almost the same as normal traffic to this model. Two points matter when comparing this table with Section 5.2. First, the reference forest is not the main model: it is trained on at most one million rows instead of the full training split, though it is tested on the same 435,121 attack records, which is why its 32,814 missed attacks are close to, but not equal to, the main model's 32,779 false negatives. Second, the counts in Table 6 are exact counts from the predictions, not estimates. So the breakdown strongly suggests that the main model's misses are also mostly Infiltration, but it does not measure this directly. Web attacks are also detected poorly, but there are only 189 of them in the test set.

Table 6: Detection per family by the reference Random Forest of the unseen-family experiment (all families, trained on ≤ 1 M rows), on the test set. Missed counts are exact.

Family	Test records	Detected	Missed
Bot	57,522	99.86%	81
Brute-force	76,112	99.99%	5
DDoS	137,982	99.90%	138
DoS	130,705	99.98%	24
Infiltration	32,611	0.65%	32,398
Web	189	11.11%	168
Total	**435,121**		**32,814**

5.4. Detection of unseen attack families (RQ3)

Table 7 and Figure 4 compare each family when it is part of training with when it is left out, with every detector scored on the same test records. Once a family is removed from training, supervised detection falls apart for most families: Bot drops from 99.9% to 0.0%, DDoS from 99.9% to 2.6% and DoS from 100.0% to 21.5%. Brute-force still reaches 50.7%, which suggests it shares some patterns with the families that remain. The high overall accuracy therefore applies to attack types the model was trained on, not to new ones.

Table 7: Detection rate (recall) per family, all columns measured on the same test records (*T*). RF seen: reference forest trained on all families. RF unseen, AD and Hybrid: family withheld from Random Forest training. AD = benign-only K-Means anomaly detector at the 1% or 0.1% calibrated threshold.

Family	Records	RF seen	RF unseen	AD 1%	Hybrid 1%	AD 0.1%	Hybrid 0.1%
Bot	57,522	99.9%	0.0%	0.1%	0.1%	0.0%	0.0%
Brute-force	76,112	100.0%	50.7%	16.7%	50.7%	0.0%	50.7%
DDoS	137,982	99.9%	2.6%	0.3%	2.9%	0.3%	2.9%
DoS	130,705	100.0%	21.5%	1.4%	22.7%	0.4%	21.8%
Infiltration	32,611	0.7%	0.7%	2.6%	3.3%	0.9%	1.6%
Web	189	11.1%	5.8%	40.7%	46.6%	25.4%	31.2%

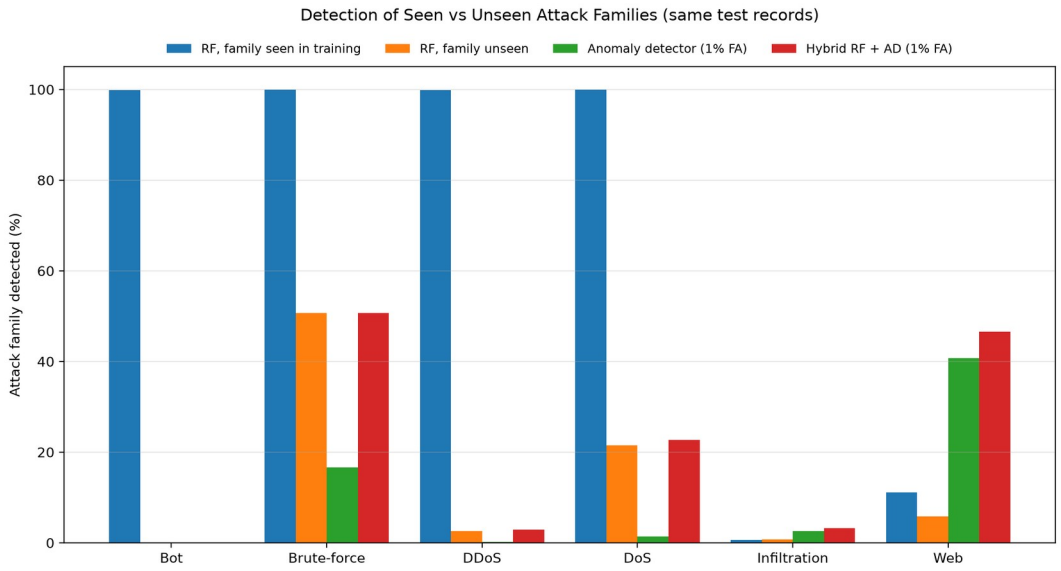


Figure 4: Detection of each attack family on the same test records: seen in training, withheld from training, and by the anomaly detector and the hybrid at the 1% threshold.

The anomaly detector, trained only on normal traffic, helps only when attack flows look very different from normal ones. It catches 40.7% of the unseen web attacks, which lifts hybrid detection of that family from 5.8% to 46.6% at the 1% threshold, and it lifts Infiltration from 0.7% to 3.3%. For Bot, DDoS and DoS it adds almost nothing (0.08%,

0.26% and 1.43%), meaning these flows fall inside the normal range as seen by a distance-based detector on standardised features. This is a negative result for this type of detector, and we report it as one. When we instead measured the unseen detectors on all of each family's pool and test records (the robustness check), no recall in Table 7 changed by more than 1.2 percentage points, so our conclusions do not depend on which records are used.

Table 8 shows the false-alarm cost on the 1,220,899 normal test flows. The thresholds, which were chosen using only normal calibration data, gave test false-alarm rates of 0.99% and 0.11% against targets of 1% and 0.1%, so the calibration carried over well to new normal traffic. At the 0.1% threshold, the hybrid adds about 1,100 false alarms per million normal flows compared with the Random Forest alone, and still raises unseen web-attack detection from 5.8% to 31.2%. At the 1% threshold the extra cost is about 9,900 per million. The forests trained without one family had normal-traffic false-alarm rates between 0.10% and 0.59%.

Table 8: False-alarm cost on benign test traffic.

Detector	False-alarm rate	Per million benign flows
Random Forest (all families)	0.477%	4,770
Anomaly detector, 1% threshold	0.990%	9,900
Hybrid, 1% threshold	1.467%	14,670
Anomaly detector, 0.1% threshold	0.110%	1,100
Hybrid, 0.1% threshold	0.587%	5,870

5.5. Prediction benchmark (RQ4)

Table 9 and Figure 5 compare the repeated benchmark with our original single run. The repeated runs were slower and varied a lot more. One million records took 11.48 ± 1.36 s on average (about 88,000 records/s), compared with 8.84 s in the single run, and at 500,000 records the standard deviation (5.93 s) was more than half the mean (9.99 s). Throughput still generally grew with batch size, from about 46,000 to 88,000 records per second, which fits with fixed per-job overheads being spread over more records; however, the means for 250,000 and 500,000 overlap within one standard deviation. The single-run numbers sit at the good end of what this VM can do, and on a shared desktop VM it is more honest to report a range from repeated runs. All timings come from one machine, so they say nothing about scaling across several machines.

Table 9: Prediction benchmark: mean ± standard deviation of three runs, compared with the original single run.

Records	Time, 3 runs (s)	Single run (s)	Throughput, 3 runs (rec/s)	Single run (rec/s)
100,000	2.26 ± 0.57	1.36	46,117 ± 11,475	73,787
250,000	4.48 ± 2.19	3.24	63,740 ± 24,312	77,203
500,000	9.99 ± 5.93	4.74	60,837 ± 27,546	105,564
1,000,000	11.48 ± 1.36	8.84	87,961 ± 10,657	113,096

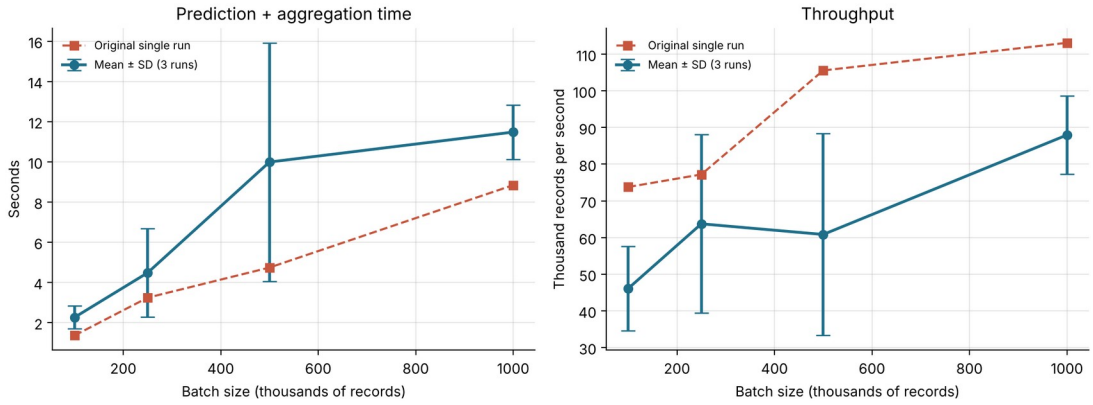


Figure 5: Repeated benchmark (mean ± SD over three runs per size) compared with the original single run.

5.6. Dashboard check and discussion

To check that the dashboard works, we uploaded a synthetic 200-row CSV with the 78 required columns. The app loaded the model from HDFS and returned 197 Benign and 3 Attack predictions. These rows have no true labels, so this only confirms that the upload-to-prediction path works; it says nothing about accuracy.

Taken together, these results change how the 97.66% accuracy should be understood. It describes attack types that were present in training; in the reference forest, almost all missed attacks belong to a single family, Infiltration; and a family missing from training is mostly not detected. An anomaly detector trained on normal traffic is not a cure-all. It helps with attacks whose flows look unusual, such as the web family, at a measurable false-alarm cost, but does little for flooding or bot traffic. A real-world detector would therefore need to be retrained regularly on newly labelled attacks, and would also need anomaly detectors chosen for the kind of behaviour they are meant to catch, with a clear limit on false alarms.

Our results have several limitations. Everything ran on one VM, so although we used Big Data tools, we did not measure scaling across machines. Half of the dataset's rows, including all of the DDoS-LOIC-HTTP traffic, were left out with the Tuesday file, and seven files may be truncated. The main model treats all attacks as one class, invalid values were set to zero, and cross-validation used a single parameter setting on a sample. The unseen-family forests were trained on at most one million rows, the anomaly

detector is a simple K-Means model, the web family has only 189 test records (831 in pool and test), and leaving out a family also leaves out traffic from particular days. Finally, the original CSE-CIC-IDS2018 files contain the labelling errors reported by Liu et al. and Cantone et al. [13, 21], and our results inherit them.

6. Conclusion

CyberSentinel is a batch Big Data pipeline for network intrusion detection. It combines HDFS storage, PySpark preprocessing, a Spark MLlib Random Forest and a Streamlit dashboard, and we applied it to 8,284,195 cleaned flows from nine CSE-CIC-IDS2018 files. On the held-out test set the model achieved 97.66% accuracy, 98.55% attack precision and 92.47% attack recall, and its training and cross-validation accuracy were within 0.05 percentage points of the test result.

Our extra analyses show what this score does and does not tell us. The file audit found an excluded 84-column file holding 49% of all rows, 59 repeated headers, and 36,307 invalid values in each of two rate features. In the reference forest of the unseen-family experiment, about 99% of the missed attacks are Infiltration, which it catches only 0.65% of the time even after training on it. When a family is left out of training, detection collapses for most families (Bot 0.0%, DDoS 2.6%). A K-Means detector trained only on normal traffic recovers part of the unseen web attacks (hybrid 46.6% at a 1% threshold and 31.2% at 0.1%, against 5.8% for the Random Forest alone, all on the same test records), at a cost of about 9,900 or 1,100 extra false alarms per million normal flows, but it adds little for flooding and bot traffic. Repeating the timing runs showed that our single-run speed figures were optimistic on the shared VM. In short, the model is very accurate on attack types it has learned but cannot reliably detect attack types it has never seen, and an anomaly detector trained on normal traffic closes only part of that gap. A practical detector therefore needs both regular retraining on newly labelled attacks and anomaly detection, with a clear limit on false alarms.

There are several directions for future work. We could fix the excluded file by dropping its four identifier columns and add its DDoS-LOIC-HTTP traffic; repeat the experiments on the corrected versions of the dataset [13, 21]; look for features or models that can tell Infiltration apart from normal traffic; try stronger anomaly detectors such as autoencoders [23] on the left-out families; add a day-based split to separate the effect of attack type from day-to-day changes; compare Random Forest with other classifiers on the same splits; and run the benchmark on an idle machine and on a real multi-node Spark cluster.

References

[1] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116. doi:10.5220/0006639801080116.

[2] J. L. Leevy, T. M. Khoshgoftaar, A survey and analysis of intrusion detection models based on CSE-CIC-IDS2018 Big Data, Journal of Big Data 7 (2020) 104. doi:10.1186/s40537-020-00382-x.

[3] Canadian Institute for Cybersecurity, CSE-CIC-IDS2018 on AWS, University of New Brunswick, dataset description and download instructions. https://www.unb.ca/cic/datasets/ids-2018.html.

[4] K. Shvachko, H. Kuang, S. Radia, R. Chansler, The Hadoop distributed file system, in: 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST), 2010, pp. 1–10. doi:10.1109/MSST.2010.5496972.

[5] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, I. Stoica, Apache Spark: A unified engine for big data processing, Communications of the ACM 59 (11) (2016) 56–65. doi:10.1145/2934664.

[6] L. Breiman, Random forests, Machine Learning 45 (2001) 5–32. doi:10.1023/A:1010933404324.

[7] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. B. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, A. Talwalkar, MLlib: Machine learning in Apache Spark, Journal of Machine Learning Research 17 (34) (2016) 1–7. https://jmlr.org/papers/v17/15-237.html.

[8] R. Sommer, V. Paxson, Outside the closed world: On using machine learning for network intrusion detection, in: 2010 IEEE Symposium on Security and Privacy, 2010, pp. 305–316. doi:10.1109/SP.2010.25.

[9] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, K. Rieck, Dos and don'ts of machine learning in computer security, in: 31st USENIX Security Symposium, 2022, pp. 3971–3988. https://www.usenix.org/conference/usenixsecurity22/presentation/arp.

[10] Inchara M P, Aysha Lintha, CyberSentinel source code, README with step-by-step reproduction instructions and data download guide, GitHub repository. https://github.com/idsaturn07/CyberSentinel.

[11] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, A. Hotho, A survey of network-based intrusion detection data sets, Computers & Security 86 (2019) 147–167. doi:10.1016/j.cose.2019.06.005.

[12] G. Engelen, V. Rimmer, W. Joosen, Troubleshooting an intrusion detection dataset: the CICIDS2017 case study, in: 2021 IEEE Security and Privacy Workshops (SPW), 2021, pp. 7–12. doi:10.1109/SPW53761.2021.00009.

[13] L. Liu, G. Engelen, T. Lynar, D. Essam, W. Joosen, Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018, in: 2022 IEEE Conference on Communications and Network Security (CNS), 2022. doi:10.1109/CNS56114.2022.9947235.

[14] G. Karataş, Ö. Demir, Ö. K. Şahingöz, Increasing the performance of machine learning-based IDSs on an imbalanced and up-to-date dataset, IEEE Access 8 (2020) 32150–32162. doi:10.1109/ACCESS.2020.2973219.

[15] S. Songma, T. Sathuphan, T. Pamutha, Optimizing intrusion detection systems in three phases on the CSE-CIC-IDS-2018 dataset, Computers 12 (12) (2023) 245. doi:10.3390/computers12120245.

[16] Apache Software Foundation, HDFS users guide – Apache Hadoop 3.3.6 documentation. https://hadoop.apache.org/docs/r3.3.6/hadoop-project-dist/hadoop-hdfs/HdfsUserGuide.html.

[17] Apache Software Foundation, Classification and regression – Spark MLlib documentation. https://spark.apache.org/docs/latest/ml-classification-regression.html.

[18] S. M. Othman, F. M. Ba-Alwi, N. T. Alsohybe, A. Y. Al-Hashida, Intrusion detection model using machine learning algorithm on Big Data environment, Journal of Big Data 5 (2018) 34. doi:10.1186/s40537-018-0145-4.

[19] R. Atefinia, M. Ahmadi, Performance evaluation of Apache Spark MLlib algorithms on an intrusion detection dataset, arXiv preprint, 2022. arXiv:2212.05269.

[20] H. Mamdouh, M. Tarek, A. Radwan, A. Saeed, A. Abdeen, M. Ashraf, K. M. Fouad, I. Abdelbaky, Apache Spark powered: Enhancing network intrusion detection system using random forest, in: 2024 6th Novel Intelligent and Leading Emerging Sciences Conference (NILES), 2024, pp. 289–294. doi:10.1109/NILES63360.2024.10753188.

[21] M. Cantone, C. Marrocco, A. Bria, On the cross-dataset generalization of machine learning for network intrusion detection, IEEE Access 12 (2024). arXiv:2402.10974.

[22] V. Chandola, A. Banerjee, V. Kumar, Anomaly detection: A survey, ACM Computing Surveys 41 (3) (2009) 15. doi:10.1145/1541880.1541882.

[23] Y. Mirsky, T. Doitshman, Y. Elovici, A. Shabtai, Kitsune: An ensemble of autoencoders for online network intrusion detection, in: Network and Distributed System Security Symposium (NDSS), 2018. doi:10.14722/ndss.2018.23204.

[24] B. Bahmani, B. Moseley, A. Vattani, R. Kumar, S. Vassilvitskii, Scalable k-means++, Proceedings of the VLDB Endowment 5 (7) (2012) 622–633. doi:10.14778/2180012.2180017.